ЛАБОРАТОРНА РОБОТА №2

Тема: Робота з ASP.NET MVC. Конфігурація проекту.
Відображення даних з БД на веб-сторінках.

Мета: Ознайомитися з архітектурою MVC, набути навичок створення головних компонентів архітектури MVC, навчитися конфігурувати підключення до БД та виконувати запити на читання даних, набути навичок роботи з представленнями (Views).

Хід роботи:

Завдання 1. Створення та конфігурація проекту

Створити проект, який реалізує архітектуру MVC, слідуючи інструкціям наведеним нижче.

Перш за все, ознайомтеся з темою вашого проекту, який ви розроблятимете протягом наступних лабораторних. Тема проекту визначається за номером по списку групи. Якщо вам не подобається тема, ви можете обрати свою, попередньо узгодивши її з викладачем. Варіанти наведено в таблиці 1.1.

Номер за списком	Тема проєкту	Короткий опис функціоналу
3	Портал вакансій	Роботодавці можуть публікувати списки вакансій, а кандидати можуть відгукуватися на вакансії та керувати своїми відгуками.

Щоб налаштувати HTTP-порт, який ASP.NET Core буде використовувати для прослуховування HTTP-запитів, внесіть зміни, показані наступному лістингу 7.4, до файлу launchSettings.json у папці Properties.

```
_ViewImports.cshtml   launchSettings.json   Program.cs
Schema: https://json.schemastore.org/launchsettings.json
1  {
2    "iisSettings": {
3      "windowsAuthentication": false,
4      "anonymousAuthentication": true,
5      "iisExpress": {
6        "applicationUrl": "http://localhost:5000",
7        "sslPort": 0
8      }
9    },
10   "profiles": {
11     "JobPortal": {
12       "commandName": "Project",
13       "dotnetRunMessages": true,
14       "launchBrowser": true,
15       "applicationUrl": "http://localhost:5000",
16       "environmentVariables": {
17         "ASPNETCORE_ENVIRONMENT": "Development"
18       }
19     }
20   }
21 }
```

Рис. 1. Лістинг launchSettings.json.

Наступним кроком створіть у вашому проєкті папки з назвами, вказаними в таблиці 1.2.

Назва	Опис
Models	Ця папка міститиме модель даних і класи, які забезпечують доступ до даних у базі даних додатка.
Controllers	Ця папка міститиме класи контролерів, які обробляють HTTP-запити.
Views	Ця папка міститиме всі файли Razor, згруповані в окремі підпапки.
Views/Home	Ця папка міститиме файли Razor, які специфічні для контролера Home.
Views/Shared	Ця папка міститиме файли Razor, які є спільними для всіх контролерів.

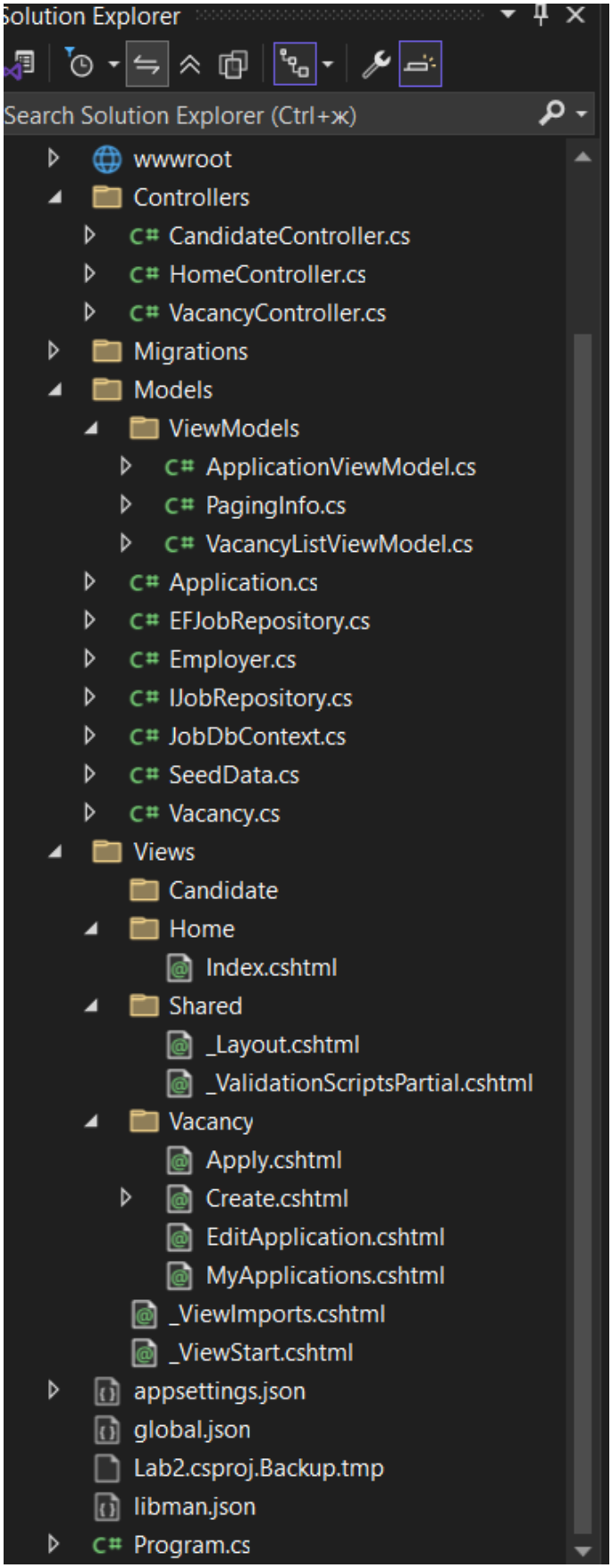

Рис. 2. Структура.

Перейдемо до налаштування сервісів та пайплайну обробки запитів. Файл Program.cs використовується для налаштування додатку ASP.NET Core.

```
_ViewImports.cshtml   Program.cs
Lab2
1  using Microsoft.EntityFrameworkCore;
2  using Lab2.Models;
3
4  var builder = WebApplication.CreateBuilder(args);
5
6  builder.Services.AddControllersWithViews();
7  builder.Services.AddDbContext<JobDbContext>(opts => {
8      opts.UseSqlServer(builder.Configuration["ConnectionStrings:JobPortalConnection"]);
9  });
10 builder.Services.AddScoped<IJobRepository, EFJobRepository>();
11
12 var app = builder.Build();
13
14 app.UseStaticFiles();
15 app.MapDefaultControllerRoute();
16
17 SeedData.EnsurePopulated(app);
18
19 app.Run();
```

Рис. 3. Лістинг Program.cs.

Перейдемо до налаштування рушія представлень (View Engine) Razor. Razor відповідає за обробку файлів перегляду з розширенням .cshtml для створення відповідей з готовим для відображення HTML. Потрібна певне початкове налаштування, Razor, щоб полегшити створення представлень.

```
_ViewImports.cshtml
Lab2
1  @using Lab2.Models
2  @using Lab2.Models.ViewModels
3  @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Рис. 4. Лістинг _ViewImports.cshtml.

Додайте файл Razor View під назвою _ViewStart.cshtml до папки Views.

```
_ViewStart.cshtml
Lab2
1  @{
2      Layout = "_Layout";
3  }
```

Рис. 5. Лістинг _ViewStart.cshtml.

Файл Razor View Start повідомляє Razor використовувати файл макета в HTML, який він створює, зменшуючи кількість дублікатів у

представленнях. Щоб створити представлення, додайте макет Razor під назвою _Layout.cshtml до папки Views/Shared.

```
_Layout.cshtml   _ViewStart.cshtml
Lab2
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <meta name="viewport" content="width=device-width" />
5       <title>Job Portal</title>
6       <link href="/lib/bootstrap/css/bootstrap.min.css" rel="stylesheet" />
7   </head>
8   <body>
9       <header class="bg-primary text-white p-3">
10          <h1>Job Portal</h1>
11      </header>
12      <div class="container mt-3">
13          @RenderBody()
14          <script src="~/lib/jquery/dist/jquery.min.js"></script>
15          <script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
16          <script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"></script>
17          @RenderSection("Scripts", required: false)
18      </div>
19      <footer class="bg-dark text-white p-3 text-center mt-3">
20          <p>&copy; 2025 Job Portal</p>
21      </footer>
22  </body>
23  </html>
```

Рис. 6. Лістинг _Layout.cshtml.

Додайте файл класу з назвою HomeController.cs у папку /Controllers і імплементуйте в ньому клас.

```
HomeController.cs   _Layout.cshtml   _ViewStart.cshtml
Lab2                                 Lab2.Controllers.Hor
1   using Lab2.Models;
2   using Lab2.Models.ViewModels;
3   using Microsoft.AspNetCore.Mvc;
4
5   namespace Lab2.Controllers
6   {
7       public class HomeController : Controller
8       {
9           private IJobRepository repository;
10          private int pageSize = 2;
11
12          public HomeController(IJobRepository repo)
13          {
14              repository = repo;
15          }
16
17          public IActionResult Index(int page = 1)
18          {
19              var vacancies = repository.Vacancies
20                  .OrderBy(v => v.VacancyId)
21                  .Skip((page - 1) * pageSize)
22                  .Take(pageSize);
23
24              var pagingInfo = new PagingInfo
25              {
26                  CurrentPage = page,
27                  ItemsPerPage = pageSize,
28                  TotalItems = repository.Vacancies.Count()
29              };
30
31              var viewModel = new VacancyListViewModel
32              {
33                  Vacancies = vacancies,
34                  PagingInfo = pagingInfo
35              };
36
37              return View(viewModel);
38          }
39      }
40  }
```

Рис. 7. Лістинг HomeController.cs.

Далі перейдемо до створення першої сутності моделі даних. Створіть файл з відповідною назвою в папці Models.

```
Application.cs | ApplicationViewModel.cs | HomeController.cs | _Layout.cshtml
Lab2 | Lab2.Models.Application
1  namespace Lab2.Models
2  {
3      public class Application
4      {
5          public long ApplicationId { get; set; }
6          public long VacancyId { get; set; }
7          public Vacancy? Vacancy { get; set; }
8          public string CandidateId { get; set; }
9          public string Resume { get; set; } = String.Empty;
10         public DateTime AppliedDate { get; set; }
11     }
12 }
```

Рис. 8. Лістинг Application.cs.

Завдання 2. Робота з ORM

Налаштувати роботу з БД, використовуючи інструкції нижче.

Тепер наш прокт містить деяке базове налаштування та може давати просту відповідь на запити. Тепер для реалізуємо роботу з даними.

Налаштування конфігурації, такі як рядки підключення до бази даних, зберігаються у файлах конфігурації JSON. Щоб визначити connection string до бази даних, яка буде використовуватися для збереження даних вашого проекту, додайте рядки, виділені у наступному лістингу, до файлу appsettings.json у теці проєкту.

```
appsettings.json | Application.cs | ApplicationViewModel.cs | HomeController.cs | _Layout.cshtml | _ViewStart.cshtml
Schema: https://json.schemastore.org/appsettings.json
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft.AspNetCore": "Warning"
6      }
7    },
8    "AllowedHosts": "*",
9    "ConnectionStrings": {
10     "JobPortalConnection": "Server=(localdb)\\MSSQLLocalDB;Database=JobPortal;MultipleActiveResultSets=true"
11   }
12 }
```

Рис. 9. Лістинг appsettings.json.

Entity Framework Core надає доступ до бази даних через клас контексту. Для проєкту SportsStore буде додано файл класу з ім'ям StoreDbContext.cs до папки Models.

```
JobDbContext.cs   appsettings.json   Application.cs   ApplicationViewModel.cs   HomeController.cs
Lab2                                                  Lab2.Models.JobDbContext
1   using Microsoft.EntityFrameworkCore;
2
3   namespace Lab2.Models
4   {
5       public class JobDbContext : DbContext
6       {
7           public JobDbContext(DbContextOptions<JobDbContext> options) : base(options) { }
8           public DbSet<Vacancy> Vacancies => Set<Vacancy>();
9           public DbSet<Employer> Employers => Set<Employer>();
10          public DbSet<Application> Applications => Set<Application>();
11      }
12  }
```

Рис. 10. Лістинг JobDbContext.cs.

Наступним кроком буде створення інтерфейсу репозиторію та його реалізації. Паттерн репозиторію є одним з найпоширеніших, і він забезпечує зручний спосіб доступу до можливостей, представлених контекстним класом бази даних. Для реалізації репозиторію для SportsStore, створимо файл з назвою IStoreRepository.cs (оберіть відповідну назву для вашого проекту) в папці Models.

```
IJobRepository.cs
Lab2                                                  Lab2.Models.IJobR
1   namespace Lab2.Models
2   {
3       public interface IJobRepository
4       {
5           IQueryable<Vacancy> Vacancies { get; }
6           void AddVacancy(Vacancy vacancy);
7           void AddApplication(Application application);
8           IQueryable<Application> Applications { get; }
9           void UpdateApplication(Application application);
10          void DeleteApplication(Application application);
11      }
12  }
```

Рис. 11. Лістинг IJobRepository.cs.

Щоб створити реалізацію інтерфейсу репозиторію, додамо файл класу з іменем EFStoreRepository.cs в папку Models і визначимо клас, показаний в лістингу:

Лістинг EFJobRepository.cs:

```
namespace Lab2.Models
{
    public class EFJobRepository : IJobRepository
    {
        private JobDbContext context;

        public EFJobRepository(JobDbContext ctx)
        {
            context = ctx;
        }

        public IQueryable<Vacancy> Vacancies => context.Vacancies;
        public IQueryable<Application> Applications => context.Applications;

        public void AddVacancy(Vacancy vacancy)
        {
            context.Vacancies.Add(vacancy);
```

```
            context.SaveChanges();
        }

        public void AddApplication(Application application)
        {
            Console.WriteLine($"Adding application:
CandidateId={application.CandidateId}, VacancyId={application.VacancyId},
Resume={application.Resume}");
            context.Applications.Add(application);
            int rowsAffected = context.SaveChanges();
            Console.WriteLine($"Rows affected: {rowsAffected}");
            if (rowsAffected > 0)
            {
                Console.WriteLine("Application successfully added to database");
            }
            else
            {
                Console.WriteLine("Failed to add application to database");
            }
        }

        public void UpdateApplication(Application application)
        {
            Console.WriteLine($"Updating application:
ApplicationId={application.ApplicationId}, Resume={application.Resume}");
            context.Applications.Update(application);
            int rowsAffected = context.SaveChanges();
            Console.WriteLine($"Rows affected: {rowsAffected}");
            if (rowsAffected > 0)
            {
                Console.WriteLine("Application successfully updated in database");
            }
            else
            {
                Console.WriteLine("Failed to update application in database");
            }
        }

        public void DeleteApplication(Application application)
        {
            Console.WriteLine($"Deleting application:
ApplicationId={application.ApplicationId}");
            context.Applications.Remove(application);
            int rowsAffected = context.SaveChanges();
            Console.WriteLine($"Rows affected: {rowsAffected}");
            if (rowsAffected > 0)
            {
                Console.WriteLine("Application successfully deleted from database");
            }
            else
            {
                Console.WriteLine("Failed to delete application from database");
            }
        }

    }
}
```

Команди Entity Framework Core виконуються з командного рядка. Відкрийте командний рядок PowerShell і виконайте команду, показану в лістингу нижче, у папці вашого проекту (для SportsStore це SportsSln/SportsStore), щоб створити клас міграції, який підготує базу даних до її першого використання.

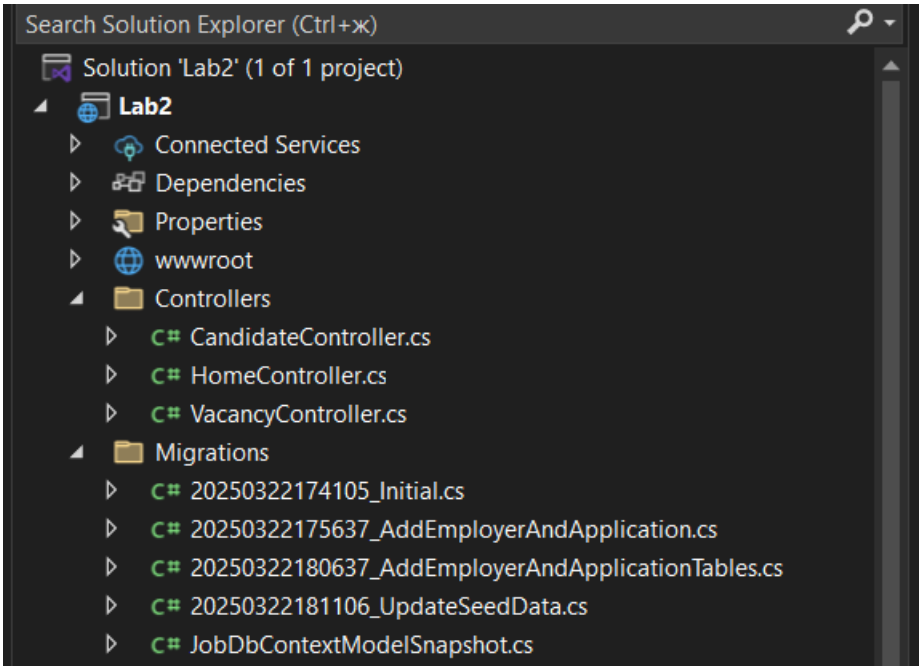

Рис. 12. Міграції.

Щоб наповнити БД даними одразу після створення, реалізуємо статичний клас SeedData в папці Models. Лістинг файлу SeedData.cs:

```
using Microsoft.EntityFrameworkCore;

namespace Lab2.Models
{
    public static class SeedData
    {
        public static void EnsurePopulated(IApplicationBuilder app)
        {
            JobDbContext context = app.ApplicationServices
                .CreateScope().ServiceProvider
                .GetRequiredService<JobDbContext>();

            if (context.Database.GetPendingMigrations().Any())
            {
                context.Database.Migrate();
                Console.WriteLine("Database migrations applied");
            }

            if (!context.Employers.Any())
            {
                context.Employers.AddRange(
                    new Employer { EmployerId = "emp1", CompanyName = "TechCorp" },
                    new Employer { EmployerId = "emp2", CompanyName = "Innovate Ltd" },
                    new Employer { EmployerId = "emp3", CompanyName = "DataWorks" }
                );
                context.SaveChanges();
                Console.WriteLine("Employers added");
            }

            if (!context.Vacancies.Any())
            {
                context.Vacancies.AddRange(
                    new Vacancy
                    {
```

```
                    Title = "Software Engineer",
                    Description = "Develop web applications",
                    Company = "TechCorp",
                    Salary = 50000,
                    EmployerId = "emp1"
                },
                new Vacancy
                {
                    Title = "Project Manager",
                    Description = "Manage IT projects",
                    Company = "Innovate Ltd",
                    Salary = 60000,
                    EmployerId = "emp2"
                },
                new Vacancy
                {
                    Title = "Data Analyst",
                    Description = "Analyze business data",
                    Company = "DataWorks",
                    Salary = 45000,
                    EmployerId = "emp3"
                }
            );
            context.SaveChanges();
            Console.WriteLine("Vacancies added");
        }
        if (!context.Applications.Any())
        {
            context.Applications.AddRange(
                new Application
                {
                    VacancyId = 1,
                    CandidateId = "candidate-id",
                    Resume = "Test resume for Software Engineer",
                    AppliedDate = DateTime.Now
                },
                new Application
                {
                    VacancyId = 2,
                    CandidateId = "candidate-id",
                    Resume = "Test resume for Project Manager",
                    AppliedDate = DateTime.Now.AddDays(-1)
                }
            );
            context.SaveChanges();
            Console.WriteLine("Test applications added");
        }
    }
}
  }
```

Завдання 3. Виведення даних на сторінці

Змініть контролер представлення для виведення даних з БД.

Лістинг файлу HomeController.cs:

```
using Lab2.Models;
using Lab2.Models.ViewModels;
using Microsoft.AspNetCore.Mvc;

namespace Lab2.Controllers
{
    public class HomeController : Controller
```

```
    {
        private IJobRepository repository;
        private int pageSize = 2;

        public HomeController(IJobRepository repo)
        {
            repository = repo;
        }

        public IActionResult Index(int page = 1)
        {
            var vacancies = repository.Vacancies
                .OrderBy(v => v.VacancyId)
                .Skip((page - 1) * pageSize)
                .Take(pageSize);

            var pagingInfo = new PagingInfo
            {
                CurrentPage = page,
                ItemsPerPage = pageSize,
                TotalItems = repository.Vacancies.Count()
            };

            var viewModel = new VacancyListViewModel
            {
                Vacancies = vacancies,
                PagingInfo = pagingInfo
            };

            return View(viewModel);
        }
    }
  }
```

Лістинг файлу Views/Home/Index.cshtml:

```
@model VacancyListViewModel

<div class="row">
    @foreach (var v in Model.Vacancies)
    {
        <div class="col-md-6 mb-3">
            <div class="card">
                <div class="card-body">
                    <h3 class="card-title">@v.Title</h3>
                    <p class="card-text">@v.Description</p>
                    <p class="card-text"><strong>Company:</strong> @v.Company</p>
                    <h4 class="card-text">@v.Salary.ToString("c")</h4>
                    <a asp-controller="Vacancy" asp-action="Apply" asp-route-
id="@v.VacancyId" class="btn btn-success">Apply</a>
                </div>
            </div>
        </div>
    }
</div>
<a asp-controller="Vacancy" asp-action="MyApplications" class="btn btn-info">View My
Applications</a>
<nav aria-label="Page navigation">
    <ul class="pagination">
        @for (int i = 1; i <= Model.PagingInfo.TotalPages; i++)
        {
            <li class="page-item @(i == Model.PagingInfo.CurrentPage ? "active" : "")">
                <a class="page-link" href="/Home/Index?page=@i">@i</a>
            </li>
```

```
        }
    </ul>
</nav>
```

Завдання 4. Реалізуйте посторінковий вивід записів.

1. У папці Models створіть підпапку ViewModels. View Model – це сутність, створена спеціально для передачі даних між контролером та представленням, яка створюється під час роботи додатку і не зберігається в БД.

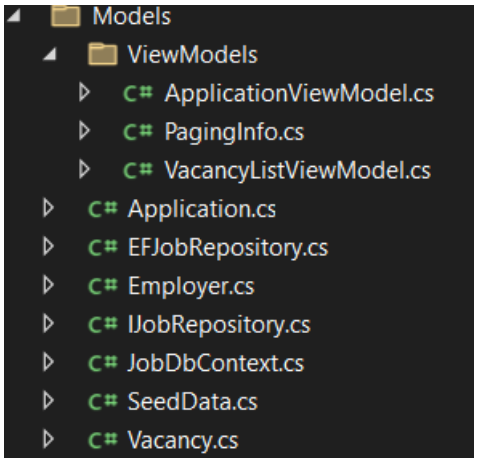

Рис. 13. Вміст підпапки.

2. В папці ViewModels створіть клас PagingInfo для збереження даних про пейджинг (кількість записів на сторінці, загальна кількість записів, кількість сторінок тощо).

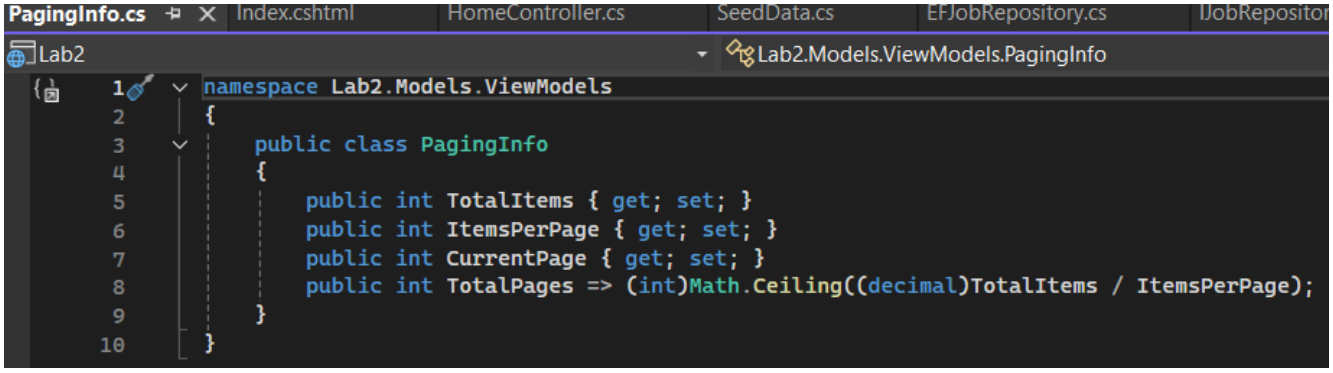

Рис. 14. Лістинг PagingInfo.cs.

3. В папці ViewModels створіть View Model для посторінкового відображення списку сутностей. Вона повинна включати в собі колекцію сутностей та інформацію про пейджинг.

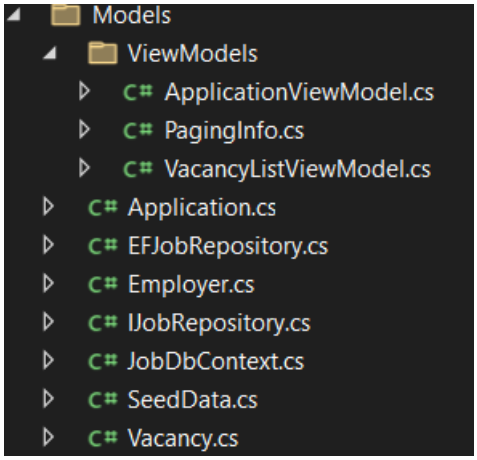

Рис. 15. Вміст підпапки.

4. Додайте підтримку пейджингу в контролер. Для цього змініть метод Index таким чином, щоб він приймав номер поточної сторінку та реалізуйте в ньому запит до БД для отримання елементів саме для цієї сторінки.

   У представлення передавайте екземпляр створеної View Model на кроці номер 3.

   Лістинг:

```
@model VacancyListViewModel

<div class="row">
    @foreach (var v in Model.Vacancies)
    {
        <div class="col-md-6 mb-3">
            <div class="card">
                <div class="card-body">
                    <h3 class="card-title">@v.Title</h3>
                    <p class="card-text">@v.Description</p>
                    <p class="card-text"><strong>Company:</strong> @v.Company</p>
                    <h4 class="card-text">@v.Salary.ToString("c")</h4>
                    <a asp-controller="Vacancy" asp-action="Apply" asp-route-
id="@v.VacancyId" class="btn btn-success">Apply</a>
                </div>
            </div>
        </div>
    }
</div>
<a asp-controller="Vacancy" asp-action="MyApplications" class="btn btn-info">View My
Applications</a>
<nav aria-label="Page navigation">
    <ul class="pagination">
        @for (int i = 1; i <= Model.PagingInfo.TotalPages; i++)
        {
            <li class="page-item @(i == Model.PagingInfo.CurrentPage ? "active" : "")">
                <a class="page-link" href="/Home/Index?page=@i">@i</a>
            </li>
        }
    </ul>
      </nav>
```

5. Адаптуйте представлення для відображення списку продуктів з View Model.
6. Додайте на сторінку елемент для вибору поточної сторінки.

Завдання 5. Стилізація

1. Встановіть фреймворк Bootstrap, виконавши команди в папці проекту.
2. Змініть базовий шаблон сторінки (_Layout.cshtml). Додайте елементи для хедера та футера.

```
_Layout.cshtml   _ViewStart.cshtml
Lab2
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <meta name="viewport" content="width=device-width" />
5       <title>Job Portal</title>
6       <link href="/lib/bootstrap/css/bootstrap.min.css" rel="stylesheet" />
7   </head>
8   <body>
9       <header class="bg-primary text-white p-3">
10          <h1>Job Portal</h1>
11      </header>
12      <div class="container mt-3">
13          @RenderBody()
14          <script src="~/lib/jquery/dist/jquery.min.js"></script>
15          <script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
16          <script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"></script>
17          @RenderSection("Scripts", required: false)
18      </div>
19      <footer class="bg-dark text-white p-3 text-center mt-3">
20          <p>&copy; 2025 Job Portal</p>
21      </footer>
22  </body>
23  </html>
```

Рис. 16. Лістинг _Layout.cshtml.

3. Стилізуйте список сутностей та пейджинг.

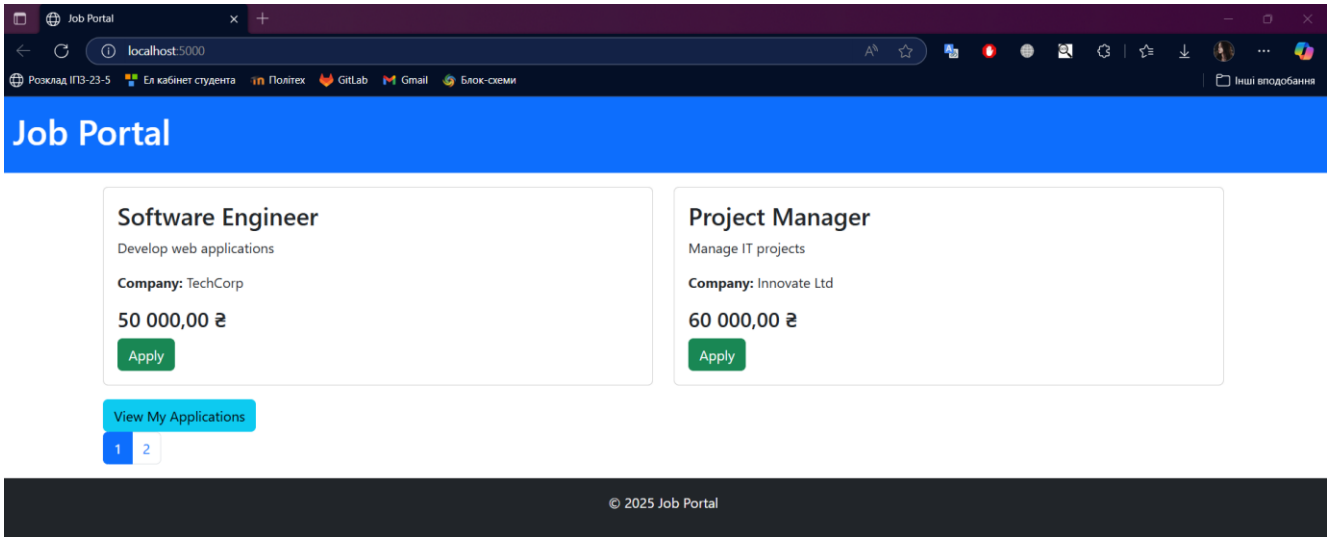

Рис. 17. Вигляд сторінки.

Висновок: в ході лабораторної роботи №2 було ознайомлено з архітектурою MVC, набуто навичок створення головних компонентів архітектури MVC, навчено конфігурувати підключення до БД та виконувати запити на читання даних, набуто навичок роботи з представ-леннями (Views).

Посилання на репозиторій:

IPZ / 2023-2027 / IPZ-23-5 / Bubon Lina / ASP.NET · GitLab